4주차 개념 정리

강화학습 개요

- 문제: 순차적으로 행동을 결정해야 하는 문제

강화학습의 개념

스키너의 강화 연구

스키너가 상자 안에 굶긴 쥐를 집어넣고 쥐가 지렛대를 누를 때마다 먹이를 제공하는 행동을 반복하였더니 쥐가 지렛대와 먹이의 관계를 학습하게 된 실험. → 인과 관계는 알 수 없지만 행동과 행동의 결과를 학습한다.

머신러닝과 강화학습

강화학습은 머신러닝의 종류 중 하나

- 지도학습: 라벨이 있는 데이터셋으로 학습 ex. 회귀분석, 분류
- 비지도 학습: 라벨 없이 주어진 데이터로만 학습 ex. 군집화
- 강화학습: 자신의 행동의 결과로 나타나는 보상을 통해 학습

그림 1.2 머신러닝은 지도학습, 비지도학습, 강화학습으로 나뉜다

에이전트 Agent

: 강화학습을 통해 스스로 학습하는 컴퓨터

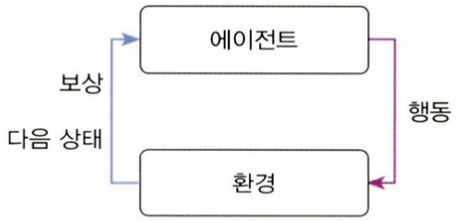


그림 1.4 에이전트와 환경의 상호작용

1. 환경에 대해 사전지식이 없는 상태에서 학습 시작
2. 자신의 놓인 환경에서 자신의 상태 인식 후 행동
3. 환경 → 에이전트에게 보상과 다음 상태 정보 제공
4. 에이전트가 스스로 좋은 행동을 학습

순차적 행동 결정 문제

강화학습은 사람처럼 환경과 상호작용하며 스스로 학습하며, 결정을 순차적으로 내려야 하는 문제, 즉 **순차적 행동 결정 문제**에 적용하면 더욱 효과적이다. 에이전트가 문제를 풀기 위해서는 문제를 수학적으로 표현하는 과정이 필요한데 그 때 사용하는 방법이 **MDP**이다.

- MDP (Markov Decision) : 순차적 행동 결정 문제를 수학적으로 정의하는 구성 요소들

순차적 행동 결정 문제의 구성 요소

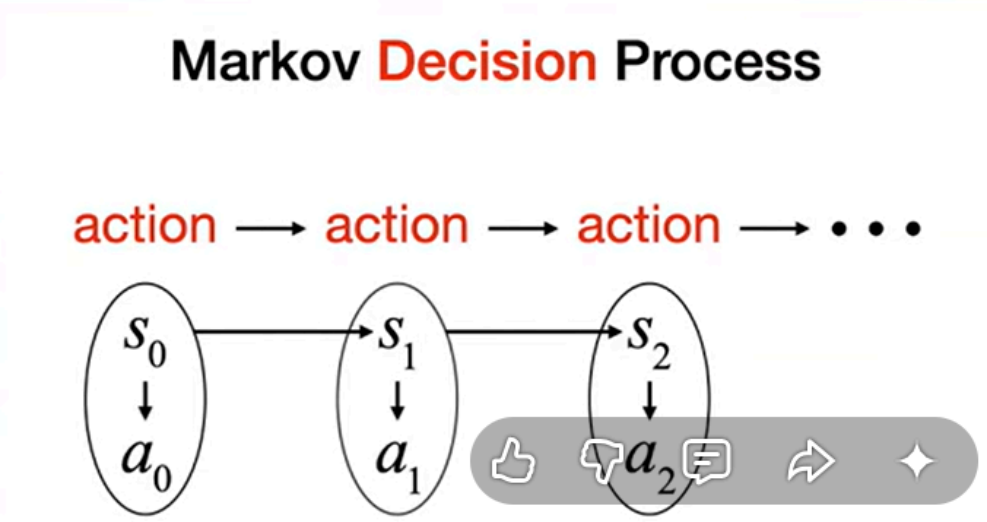
1. **상태 state**
   : 에이전트가 상황을 판단해서 행동을 결정하기 위해 제공되는 정보, 현재의 상태
2. **행동 action**
   : 에이전트가 어떠한 상태에서 취할 수 있는 행동
3. **보상 reward**
   : 에이전트가 자신의 행동을 평가하고 좋은 행동을 학습하게 하는 정보
   - 에이전트에 속하지 않는 환경의 일부로, 강화학습의 목표는 얻는 보상들의 합을 최대화 하는 것
4. **정책 policy**
   : 에이전트가 순차적 행동 결정 문제에서 구해야 할 답

에이전트는 상태를 기반으로 행동을 하고, 그에 환경으로부터 얻어지는 보상을 기반으로 학습하여 최적의 정책을 얻어낸다!

MDP와 벨만 방정식

MDP(Markov Decision Process)

: 순차적으로 행동을 결정해야 하는 문제를 수학적으로 표현



MDP의 구성 요소

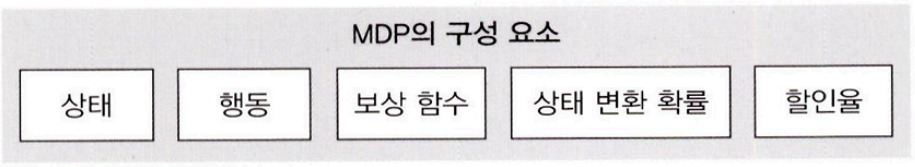


그림 2.1 MDP의 구성 요소

1.상태

S: 에이전트가 관찰 가능한 상태의 집합

그리드월드 예시에서의 상태의 개수는 유한하며 다음과 같이 표현 가능하다.

MDP에서 상태는 시간에 따라 확률적으로 변하며, 확률변수가 된다.

시간 t에서의 상태 S_t가 어떤 상태 s다.

$$S_t = s$$

2.행동

에이전트가 상태 S_t에서 할 수 있는 가능한 행동의 집합은 A이다.

- 시간 t에서의 행동 a

$$A_t = a$$

3.보상 함수

: 에이전트가 학습가능한 유일한 정보로서, 환경이 에이전트에게 제공한다.

- 시간 t에서 상태가 $S_t = s$이고 행동이 $A_t = a$일 때 에이전트가 받을 보상

$$r(s, a) = E[R_{t+1}|S_t = s, A_t = a]$$

> ❓ 보상 함수를 기댓값으로 표현하는 이유는 뭘까?
> '|' 뒤의 부분들은 현재의 보상함수의 조건 혹은 상태와 행동이다. 환경에 따라서 같은 상태에서 같은 행동을 하였더라도 다른 보상이 주어질 수 있기 때문에 기댓값으로 표현하는 것이다.

4.상태 변환 확률

: 상태 s에서 행동 a를 취했을 때 다른 상태 s'에 도달할 확률

$$P_{ss}^{a} = P[S_{t+1} = s|S_t = s, A_t = a]$$

5.할인율

: 미래에 받을 보상의 가치를 현재의 가치로 환산한 비율

$$\gamma \in [0, 1]$$

수식 2.13 할인율의 정의

- 현재의 시간 t에서 시간 k가 지난 후에 받을 보상 R_{t+k}의 가치는 다음과 같으며, 먼 미래에 받을 보상일수록 그 가치는 더 작아진다.

$$\gamma^{k-1} R_{t+k}$$

수식 2.14 할인율을 고려한 미래 보상의 현재 가치

정책

: 모든 상태에서 에이전트가 할 행동

$$\pi(a \mid s) = P[A_t = a \mid S_t = s]$$

수식 2.15 정책의 정의

- 강화학습을 통해 에이전트는 수많은 정책 중 최적 정책을 선택해야 한다.
  - +가치 함수를 최대화하는 값이 최적 정책

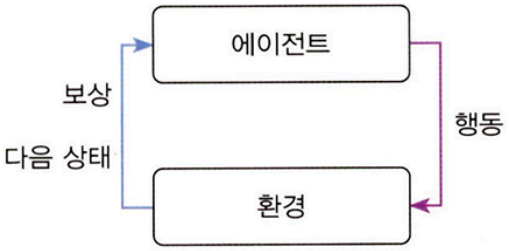


그림 2.9 에이전트와 환경의 상호작용

가치함수

시간 t에 수행한 행동으로 받는 보상 = R_{t+1}

→ 행동에 대한 보상은 그 다음 스텝에 받게 된다!

- 가치함수: 어떠한 상태에 도달하면 받을 것이라고 예상되는 값

❓ 각 시점에서의 보상에 대한 단순합을 하면 어떤 문제가 있을까?

1. 미래에 대한 보상에 대한 할인율 적용을 하지 않는다.
2. 큰 보상을 한 번에 받는 것과 분배해서 받는 것을 구분하지 못한다.
3. 시간이 무한대일 때, 작은 보상과 큰 보상 모두 무한으로 발산하기 때문에 둘을 구분하지 못한다.

⇒ 에이전트는 보상의 단순합으로는 가치 판단이 어렵다.
⇒ 그래서 도입한 개념이 **할인율**이다.

할인율

: 미래의 보상을 현재의 보상으로 변환하기 위해 미래의 보상에 곱해주는 값

반환값

: 할인율을 적용하여 구한, 에이전트가 실제로 받는 보상의 합

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + ...$$

→ 반환값에 대한 기댓값으로 가치를 판단하며, 이것이 바로 **가치함수**이다.

가치함수

$$v(s) = E[G_t | S_t = s]$$

$$\mathrm{v}(\mathrm{s}) = \mathrm{E}[\mathrm{R}_{t+1} + \gamma \mathrm{R}_{t+2} + \gamma^2 \mathrm{R}_{t+3} \cdots \mid \mathrm{S}_t = \mathrm{s}]$$

수식 2.22 앞으로 받을 보상에 대한 기댓값인 가치함수

$$\mathrm{v}(\mathrm{s}) = \mathrm{E}[\mathrm{R}_{t+1} + \gamma(\mathrm{R}_{t+2} + \gamma \mathrm{R}_{t+3} \cdots) \mid \mathrm{S}_t = \mathrm{s}]$$
$$\mathrm{v}(\mathrm{s}) = \mathrm{E}[\mathrm{R}_{t+1} + \gamma \mathrm{G}_{t+1} \mid \mathrm{S}_t = \mathrm{s}]$$

수식 2.23 반환값으로 나타내는 가치함수

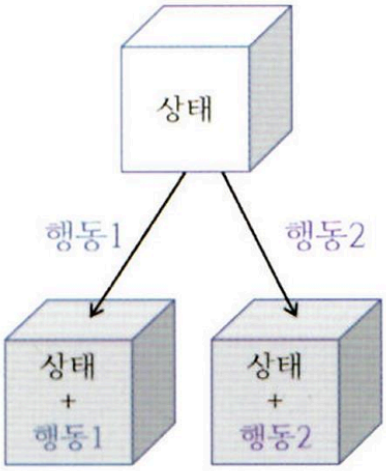
위의 수식은 정책을 고려하지 않고 계산한 수식이다. 하지만 다음 상태로 넘어갈 때 에이전트가 하는 행동이 바로 정책이므로 **정책에 따라 계산**해야 하며, 이는 바로 아래의 수식인 **벨만 기대 방정식**이 된다.

$$\mathrm{v}_\pi(\mathrm{s}) = \mathrm{E}_\pi[\mathrm{R}_{t+1} + \gamma \mathrm{v}_\pi(\mathrm{S}_{t+1}) \mid \mathrm{S}_t = \mathrm{s}]$$

수식 2.25 정책을 고려한 가치함수의 표현

큐함수

- **상태 가치함수**: 상태가 입력으로 들어오면 그 **상태**에서 앞으로 받을 보상의 합을 출력
- **행동 가치함수(=큐함수)**: 어떤 상태에서 어떤 **행동**이 얼마나 좋은지 그 가치를 알려주는 함수
    - 상태, 행동이라는 변수를 가지며 $q_\pi(s, a)$로 나타낸다.



흰상자: 상태, 회색 상자: 행동 상태

🔎 **가치함수와 큐함수의 관계**

1. 각 행동을 했을 때 받을 보상인 큐함수 $q_\pi(s,a)$를 $\pi(a|s)$에 곱한다.
2. 모든 행동에 대해 큐함수와 $\pi(a|s)$를 곱한 값을 더하면 가치함수가 된다.

$$v_\pi(s) = \sum_{a \in A} \pi(a \mid s)\, q_\pi(s,a)$$

수식 2.26 가치함수와 큐함수 사이의 관계식

- 강화학습에서 에이전트가 행동을 선택하는 기준으로 보통 가치함수보다 큐함수를 사용한다.

$$q_\pi(s,a) = E_\pi[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a]$$

수식 2.27 큐함수의 정의

조건문 뒤에 행동이 들어간다는 부분이 가치함수와의 차이점

벨만 기대 방정식

- 벨만 방정식: 현재 상태의 가치함수와 다음 상태의 가치함수의 관계를 나타낸 방정식
- 벨만 **기대** 방정식이라고 칭하는 이유는 식에 기댓값이 들어가기 때문!

예시) 1을 100번 더하는 문제를 푸는 두 가지 방법

$$1 + 1 + 1 + ... + 1 = 100$$

```
X=0
for i in range(100):
    X=X+1
```

- 벨만 방정식으로 가치함수를 계산하는 것은 후자의 방식과 같다.
  - 한 번에 모든 것을 계산 (X)
  - 값을 변수에 저장하고 루프를 도는 계산(O)

$$v_\pi(s) = E_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s]$$

수식 2.28 벨만 기대 방정식

→ 현재 가치함수 값을 업데이트

기댓값은 어떻게 계산할까?

- 기댓값에는 어떠한 행동을 할 확률(정책)과 그 행동 수행 시 어떠한 상태로 갈 확률(상태 변환확률)을 포함하고 있으므로 다음과 같이 계산 가능하다.

$$v_\pi(s) = \sum_{a \in A} \pi(a \mid s)\left(r_{(s,a)} + \gamma \sum_{s' \in S} P^a_{ss'} v_\pi(s')\right)$$

수식 2.32 계산 가능한 벨만 방정식

그리드월드로 보는 예제

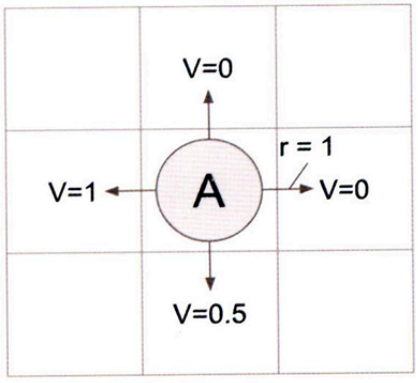


그림 2.17 그리드월드에서 가치함수의 업데이트

표 2.1 벨만 기대 방정식의 계산

1	행동 = 상	$0.25 \times (0 + 0.9 \times 0) = 0$
2	행동 = 하	$0.25 \times (0 + 0.9 \times 0.5) = 0.1125$
3	행동 = 좌	$0.25 \times (0 + 0.9 \times 1) = 0.225$
4	행동 = 우	$0.25 \times (1 + 0.9 \times 0) = 0.25$
총합	기댓값 $= 0 + 0.1125 + 0.225 + 0.25 = 0.5875$	

- A: 에이전트의 현재 상태
- 에이전트의 초기 정책: 4가지 가능한 행동을 25%의 확률로 선택

1. 각 행동에 대해 그 행동을 할 확률을 고려
2. 각 행동을 했을 때 받을 보상 고려
3. 다음 상태의 가치함수 고려

⇒ 업데이트를 통해 현재의 정책을 따라갔을 경우 에이전트가 얻을 리턴값을 구한다.

벨만 최적 방정식

- **참 가치함수**: "어떠한 정책"을 따라 행동했을 때 받게 되는 보상에 대한 참값
- **최적 가치함수**: 수많은 정책 중 가장 높은 보상을 얻게 되는 정책을 따랐을 때의 가치함수

> 어떤 정책이 더 좋은 정책일까?

→ 답은 가치함수에 있다!: 모든 정책 중 **가장 큰 가치함수**를 주는 정책이 최적 정책

$$v_*(s) = \max_{\pi}[v_\pi(s)]$$

수식 2.36 최적의 가치함수

$$q_*(s,a) = \max_{\pi}[q_\pi(s,a)]$$

수식 2.37 최적의 큐함수

$$\pi_*(s,a) = \begin{cases} 1 & \text{if } a = \text{argmax}_{a \in A} q_*(s,a) \\ 0 & \text{otherwise} \end{cases}$$

수식 2.38 최적 정책

$$v_*(s) = \max_a E[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a]$$

수식 2.40 벨만 최적 방정식

$$q_*(s,a) = E[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') \mid S_t = s, A_t = a]$$

수식 2.41 큐함수에 대한 벨만 최적 방정식

다이내믹 프로그래밍

명시적 정책 vs 내재적 정책

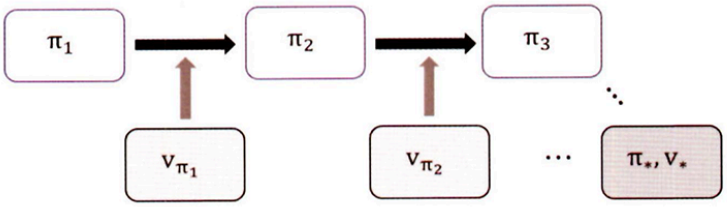


그림 3.17 정책 이터레이션에서 정책과 가치함수가 발전해나가는 과정

벨만 최적 방정식과 가치 이터레이션

- 다이내믹 프로그래밍: 큰 문제에 작은 문제들이 중첩된 경우, 큰 문제 → 작은 문제로 쪼개고 작은 문제들의 결과를 상호 이용해 푸는 것
- 벨만 기대 방정식 → 정답: 참 보상

$$v_\pi(s) = E_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s]$$

수식 3.12 벨만 기대 방정식을 통해 구하는 것은 현재 정책에 대한 참 가치함수다

- 벨만 최적 방정식 → 정답: 최적 가치 함수

$$v_*(s) = \max_a E[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a]$$

수식 3.13 벨만 최적 방정식을 통해 구하는 것은 최적 가치함수다

- `max`를 취하기 때문에 정책의 값 고려X

- **가치 이터레이션**: 벨만 최적 방정식을 이용하여 최적 가치함수를 반복적인 방법으로 계산하는 것